

HelixSpecifier

HelixSpecifier

Распрацоўка на аснове спецыфікацый, якая сама маштабуе сваю цырымонію да аб'ёму працы.

Зводка

HelixSpecifier — рухавік Go, які аб'ядноўвае тры метадалогіі распрацоўкі: спецыфікацыйна-арыентаваны працоўны працэс SpecKit, дысцыпліну TDD ад Superpowers і жыццёвы цыкл этапных мэтаў GSD — у адзіны адаптыўны паток. Ён класіфікуе кожную задачу паводле ўзроўню складанасці і адпаведна маштабуе аб'ём працэсу.

Кароткае апісанне

HelixSpecifier — рухавік аб'яднанай спецыфікацыйна-арыентаванай распрацоўкі для агентаў AI. Ён інтэгруе SpecKit, Superpowers і GSD, класіфікуе працу паводле ўзроўню складанасці, праводзіць фазы спецыфікацыі з падтрымкай дыскусій, выконвае мінімальны суадносіны тэстаў да кода і вучыцца на кожным завершаным працоўным патоку.

Падрабязнае апісанне

HelixSpecifier — рухавік спецыфікацыйна-арыентаванай распрацоўкі (SDD), напісаны на Go (модуль `digital.vasic.helixspecifier`) і створаны як кампанент ансамбля HelixAgent AI. Ён аб'ядноўвае тры практыкі распрацоўкі, якія звычайна існуюць у трох асобных інструментах — і трох розных ментальных мадэлях — у адзіны адаптыўны працоўны працэс: сяміфазны працэс SDD ад SpecKit (Constitution, Specify, Clarify, Plan, Tasks, Analyze, Implement), дысцыпліну TDD ад Superpowers з паралельным выкананнем падзадач агентамі і кіраванне жыццёвым цыклам этапных мэтаў ад GSD. Кожны з

трох кампанентаў працягвае выконваць сваю функцыю, але менавіта рухавік забяспечвае іх працу як адзінага ўзгодненага патоку, а не ручнога «зшывання» канвеера.

Цэнтральная ідэя — *адаптыўная цырымонія*: рухавік класіфікуе ўваходныя задачы паводле ўзроўню складанасці і маштабуе аб'ём працэсу такім чынам, каб аднарадковае выпраўленне не праходзіла праз той жа грувасткі рытуал, што і распрацоўка вялікай функцыі, а вялікая функцыя не праходзіла з той жа лёгкасцю, што і выпраўленне апіскі. На гэтай аснове пабудавана дзесяць ключавых функцый: паралельнае выкананне задач з абмежаванай канкурэнтнасцю, машынна-чытальны «Constitution як код», які аўтаматычна прымушае выконваць абавязковыя правілы, «Nyquist TDD», што адсочвае і падтрымлівае мінімальнае суадносіны тэстаў да рэалізацыі, шматраундовыя дыскусіі некалькіх агентаў для ўдасканалення спецыфікацый, адаптыўнае навучанне навыкам, аналіз існуючага кода, прадказанне спецыфікацый на аснове гістарычных шаблонаў, перанос ведаў паміж праектамі, дынамічная карэкцыя цырымоніі падчас выканання, а таксама пастаянная памяць спецыфікацый з семантычным пошукам.

Рухавік выкарыстоўваецца як модуль Go — праз `go get` ці лакальную дырэктыву `replace` — за невялікім інтэрфейсам API: рэгіструюцца тры кампаненты, маштабавальнік цырымоніі і памяць спецыфікацый, вызначаецца складанасць задачы, пасля чаго запускаяецца поўны працоўны працэс і вяртаецца вынік з ацэнкай якасці. Інтэрфейс просты, але аркестроўка за ім — не. Як і астатнія кампаненты сямейства Helix, ён распрацоўваецца ў рэжыме анты-блефу: у працэсе працы запускаяецца сістэма праверкі, якая тэсціруе рэальны код, а не макеты.

Чаму мы яго стварылі

Спецыфікацыйна-арыентаваная распрацоўка, строгая дысцыпліна TDD і кіраванне этапнымі мэтамі звычайна з'яўляюцца трыма асобнымі практыкамі з трыма асобнымі інструментамі. HelixSpecifier быў створаны для таго, каб агент AI (HelixAgent) мог выконваць усе тры кампаненты як адзіны ўзгоднены, самамаштабавальны працоўны працэс, а не збіраць іх уручную.

Змест

Чаму гэта змяняе гульні

Гэта аўтаматычна робіць працэс прапарцыйным працы. Каманды звычайна ўтыкаюцца ў адзін з двух дрэнных варыянтаў: цяжкія цырымоніі для ўсяго (бяспечна, але павольна і таёмна выклікае незадаволенасць) ці ніякіх цырымоній (хутка, пакуль не перастае быць хутка). HelixSpecifier здымае гэтую дилему, падганяючы цырымонію пад класіфікаваны аб'ём кожнага задання і пераналаджваючы яе ў працэсе выканання, калі праца пачынае выяўляць свае асаблівасці. Магчымасць, якая раней была непрактычнай, — гэта працэс, што самастойна выбірае аптымальны памер для кожнага задання, а на версе гэтага — рашэнні па спецыфікацыях, падмацаваныя шматраундовымі дыскусіямі некалькіх агентаў з ацэнкай пазіцый замест першага здагаду аднаго агента.

Што новага

- **Адаптыўная цырымонія** — узровень працэсу кіруецца рэальнымі метрыкамі якасці і карэктуюцца на ходу, а не фіксуецца загадзя.
- **Найквістаўскае TDD** — бар'ер суадносін тэстаў да рэалізацыі (мінімум 2х), што запазычвае логіку тэарэмы Найквіста аб дыскрэтызацыі: каб дакладна адлюстравалі паводзіны, трэба вымяраць іх значна часцей за іх частату, таму тэсты павінны перавышаць код, які яны пакрываюць.
- **Архітэктурны дыскусіі** — шматраундовы, шматраундовы працэс удасканалення спецыфікацый, дзе прапановы выстаўляюцца, ацэньваюцца і ўзгадняюцца, замяняючы адзіную думку на супрацьстаянне меркаванняў.
- **Прагностычны спецыфікацыі і перанос паміж праектамі** — рухавік аналізуе назапашаныя патокавыя даныя, каб прадбачыць спецыфікацыі і пераносіць цяжка дасягнутыя веды з аднаго праекта ў наступны.
- **Constitution як код** — абавязковыя правілы праекта робяцца машынначытальнымі і прымусява выконваюцца рухавіком, а не застаюцца на сумленні рэцэнзентаў.

Галоўныя тэхнічныя праблемы і як мы іх вырашылі

- **Аб'яднанне трох метадалогій без іх канфлікту** — SpecKit, Superpowers і GSD кожная лічыць, што менавіта яна кантралюе працоўны працэс. Вырашана з дапамогай рухавіка-зліцця, які рэгіструе кожную калону за агульным інтэрфейсам і кіруе імі праз адзіны жыццёвы цыкл патоку,

каб яны складаліся ў адзіны працэс замест трох сутыкнёных.

- **Вызначэнне, колькі працэсу насамрэч патрабуе дадзенае заданне** — калі пераацаніць, усё павольна павузае; калі недаацаніць — рызыкаўная праца адпраўляецца без праверкі. Вырашана з дапамогай класіфікатара аб'ёму працы, які вызначае маштаб задання і падае яго на маштабатар цырымоніі, што дынамічна карэктуюе ўзровень працэсу па меры выканання.
- **Захаванне высокай якасці спецыфікацый без чалавека-кантралёра на кожным рашэнні** — вырашана заменай аднаразовых спецыфікацый на дыскусію з удасканаленнем, дзе агенты ацэньваюць канкуруючыя пазіцыі на працягу некалькіх раўндаў, а таксама абавязковым выкананнем суадносін Найквістаўскага TDD, каб рэалізацыя не абыходзіла тэсты.

Тэхнічны стэк

- **Go** — абраны, каб рухавік пастаўляўся як адзіны бінарны файл для імпарту без дадатковых залежнасцей падчас выканання; яго мадэль канкурэнтнасці дазваляе апрацоўваць заданні ў абмежаваным паралельным рэжыме і праводзіць шматраундовыя дыскусіі агентаў без галаўной болі з ніткамі.
- **logrus** — структураванае лагіраванне, якое пранізвае рухавік і ўсе тры калоны, каб рашэнні патоку (класіфікацыя, змены цырымоніі, вынікі дыскусій) заставаліся зразумелымі постфактум.
- **Калонна SpecKit** — сяміфазавы працэс распрацоўкі на аснове спецыфікацый (Constitution → Спецыфікацыя → Удакладненне → План → Заданні → Аналіз → Рэалізацыя), што забяспечвае дысцыплінаваны каркас пераўтварэння спецыфікацыі ў код.
- **Калонна Superpowers** — дысцыпліна TDD з паралельным выкананнем субагентаў, што забяспечвае строгасць тэставага падыходу і разгалінаванне, якое захоўвае рэалізацыю сумленнай і хуткай.
- **Калонна GSD** — кіраванне этапамі і жыццёвым цыклам, што надае патоку адчуванне «зроблена» і прасоўванне па стадыях.
- **Сховішча памяці спецыфікацый** — пастаянны, семантычна пошукавы індэкс мінулых спецыфікацый, аснова, што робіць магчымымі прагнозныя спецыфікацыі і перанос ведаў паміж праектамі замест таго, каб кожны раз пачынаць з нуля.

Змест

Заўвагі аб статусе і сумленнасці

- **Статус: бэта-версія.** Выкарыстоўваецца як складнік модуля Go у межах HelixAgent.
- **Ліцэнзія: пакуль не вызначана.** Ліцэнзія не была выяўлена праз GitHub API — НЕПРАВЕРАНА / не дэкларавана.
- Адлюстраваная назва “HelixSpecifier” адпавядае рэпазіторыю specifier.

Прыярытэтны ўзровень: Helix-асноўны.